

RESEARCH ARTICLE | NOVEMBER 02 2022

Learning unseen coexisting attractors

Daniel J. Gauthier  ; Ingo Fischer  ; André Röhm 



Chaos 32, 113107 (2022)

<https://doi.org/10.1063/5.0116784>



Articles You May Be Interested In

Model-free inference of unseen attractors: Reconstructing phase space features from a single noisy trajectory using reservoir computing

Chaos (October 2021)

Effect of temporal resolution on the reproduction of chaotic dynamics via reservoir computing

Chaos (June 2023)

Learning beyond experience: Generalizing to unseen state space with reservoir computing

Chaos (October 2025)



Chaos

**Special Topics Open
for Submissions**

[Learn More](#)

Learning unseen coexisting attractors

Cite as: Chaos 32, 113107 (2022); doi: 10.1063/5.0116784

Submitted: 29 July 2022 · Accepted: 10 October 2022 ·

Published Online: 2 November 2022



View Online



Export Citation



CrossMark

Daniel J. Gauthier,^{1,2,a)} Ingo Fischer,³ and André Röhm⁴

AFFILIATIONS

¹Department of Physics, The Ohio State University, 191 West Woodruff Ave., Columbus, Ohio 43210, USA

²ResCon Technologies LLC, P.O. Box 21229, Columbus, Ohio 43221, USA

³Instituto de Física Interdisciplinar y Sistemas Complejos, IFISC (CSIC-UIB), Campus Universitat Illes Balears, E-07122 Palma de Mallorca, Spain

⁴Department of Information Physics and Computing, Graduate School of Information Science and Technology, The University of Tokyo, 7-3-1 Hongo, Bunkyo-ku, Tokyo 113-8656, Japan

^{a)}Author to whom correspondence should be addressed: gauthier.51@osu.edu

ABSTRACT

Reservoir computing is a machine learning approach that can generate a surrogate model of a dynamical system. It can learn the underlying dynamical system using fewer trainable parameters and, hence, smaller training data sets than competing approaches. Recently, a simpler formulation, known as next-generation reservoir computing, removed many algorithm metaparameters and identified a well-performing traditional reservoir computer, thus simplifying training even further. Here, we study a particularly challenging problem of learning a dynamical system that has both disparate time scales and multiple co-existing dynamical states (attractors). We compare the next-generation and traditional reservoir computer using metrics quantifying the geometry of the ground-truth and forecasted attractors. For the studied four-dimensional system, the next-generation reservoir computing approach uses $\sim 1.7\times$ less training data, requires $10^3\times$ shorter “warmup” time, has fewer metaparameters, and has an $\sim 100\times$ higher accuracy in predicting the co-existing attractor characteristics in comparison to a traditional reservoir computer. Furthermore, we demonstrate that it predicts the basin of attraction with high accuracy. This work lends further support to the superior learning ability of this new machine learning algorithm for dynamical systems.

Published under an exclusive license by AIP Publishing. <https://doi.org/10.1063/5.0116784>

Reservoir computing is a machine learning (ML) approach that is well suited to learning dynamical systems using a short sample of time-series data. It learns the underlying dynamical system so that the trained model serves as a surrogate or a “digital twin” of the system, and it can reproduce behaviors not contained in the training data set. For chaotic systems, it can reproduce the associated strange attractor and other characteristics. Surprisingly, the reservoir computer (RC) can predict all attractors of a multi-attractor dynamical system even when it has been trained on time-series data generated by the system when it resides on or in the vicinity of a single attractor. Here, we show that a variant machine learning algorithm, known as a next-generation reservoir computer, can reproduce co-existing attractors and the associated basin of attraction using smaller training data sets and with higher accuracy than a traditional reservoir computer.

I. INTRODUCTION

As human-built dynamical systems become more complex and autonomous, such as advanced propulsion systems or

autonomous cars, there is a growing need to develop surrogate models that can be used to rapidly forecast future behaviors, predict boundaries of operation, control the system, or indicate required maintenance, for example. These surrogate models are often known as “digital twins.”¹ Ideally, they can be traced back to the underlying physics governing the dynamics, be evaluated faster than in real time for control applications, and incorporate current observations to improve the model performance.

While many machine learning (ML) algorithms can excel at this task, reservoir computing has emerged as a leading candidate. A reservoir computer (RC) is an artificial neural network with an input layer through which time-series data is input, a recurrent network (the “reservoir”) where each neuron in the network has its own dynamical behavior, and an output layer that is trained for the desired task. Notably, only the weights of the output layer are trained, often by supervised learning, whereas the weights connecting the input layer to the reservoir and the recurrent connection weights are assigned randomly before training begins and are held fixed thereafter.^{2,3}

The RC excels as a digital twin because of its ability to perform accurate prediction while requiring smaller training data set sizes in comparison to other approaches.^{4,5} Also, the model can be trained using only experimental data⁶ or a hybrid version can be realized that uses both experimental observations and physical model predictions.⁷ The RC paradigm maps well onto physical devices, which can perform the equivalent analog computations to reduce power consumption or increase prediction speed.⁸

Recently, a new ML algorithm was introduced, known as next-generation reservoir computing (NG-RC), that is mathematically equivalent to a traditional RC and shares its attributes,⁹ but is much simpler. Specifically, the fading memory of the traditional reservoir computer arising from the finite response time of the neurons and the recurrent topology is replaced by time-delay copies of the incoming data.¹⁰ To be theoretically equivalent to a traditional RC, it requires time-delay copies of the incoming data extending to infinitely far in the past^{11,12} but we find that truncating to include the most recent past data is enough to give state-of-the-art performance. Also, the nonlinearity of the neurons in a traditional RC is moved to the output layer, which is a sum of nonlinear functionals of the time-delay data in the NG-RC approach. Often, polynomial functionals work well, which can be motivated by theories of universal approximators of dynamical systems.¹¹ The high-weighted nonlinear functionals can be traced back to the physical model of the system, thus peeling back the “black box” nature of many ML algorithms.

Another important attribute of the NG-RC approach is that there are no random weights connecting the input layer to the reservoir or within the reservoir as there is in a traditional RC; in fact, there is no longer a recurrent network.¹⁰ Overall, the NG-RC has fewer trainable parameters because the output layer has fewer elements than a traditional RC and, hence, smaller training data set sizes are required. Furthermore, the NG-RC has no random weights whose statistical properties need to be optimized as in a traditional RC, thus reducing the computation time needed to obtain high performance. The current downside of the approach is that some insight is needed to design a compact output feature vector.

Here, we use an NG-RC to create a digital twin of a dynamical system with multiple co-existing attractors and, by coincidence, disparate time scales. We generate synthetic training data by integrating the differential equations describing the system and using initial conditions that direct the system to only one of the attractors for the training data set. We then use the trained NG-RC model to predict the other co-existing attractors (unseen during training) of the system by changing the initial conditions.

This problem is particularly challenging: We previously used a traditional RC for the same problem¹³ and found modest prediction ability after spending substantial computing time to optimize the algorithm metaparameters. The NG-RC algorithm obtains an accurate solution requiring much less effort to optimize the few remaining metaparameters. The purpose of this paper is to report our approach and document the accuracy of the ML algorithm.

In Sec. II, we describe the dynamical system introduced by Li and Sprott,¹⁴ which is one of the simplest dynamical systems that display co-existing attractors. In Sec. III, we briefly describe the NG-RC and performance metrics and then give our results in Sec. IV. We end in Sec. V with a discussion and point to future directions.

II. A DYNAMICAL SYSTEM WITH CO-EXISTING ATTRACTORS

We focus on systems described by d autonomous nonlinear differential equations of the form

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}), \quad (1)$$

with initial condition $\mathbf{x}_0$. Here, $\dot{\mathbf{x}} \equiv d\mathbf{x}/dt$ and $\mathbf{f}$ is referred to as the vector field.

When the time-series data generated by Eq. (1) is sampled at discrete times $\{t_n, t_{n-1}, t_{n-2}, \dots\}$ with step $dt = t_n - t_{n-1}$, it is appropriate to consider the flow $\mathbf{F}$ of the dynamical system, which is an evolution function that steps it from one discrete point to the next through the relation

$$\mathbf{x}_{n+1} = \mathbf{F}(\mathbf{x}_n). \quad (2)$$

Formally, the flow can be found by integrating Eq. (1) as

$$\mathbf{F}(\mathbf{x}_n) = \mathbf{x}_n + \int_{t_n}^{t_{n+1}} \mathbf{f}(\mathbf{x}) ds. \quad (3)$$

We consider the $d = 4$ system introduced by Li and Sprott¹⁴ given by

$$\dot{x} = -x + y, \quad (4)$$

$$\dot{y} = -xz + u, \quad (5)$$

$$\dot{z} = xy - a, \quad (6)$$

$$\dot{u} = -by, \quad (7)$$

where $\mathbf{x} = [x, y, z, u]^T$, T is the transpose, and a and b are parameters. Despite its simplicity, Eqs. (4)–(7) show a variety of behaviors including a large separation of time scales, periodic, quasi-periodic, and chaotic dynamics depending on the parameter values, and they do not have any stable or unstable steady states. The system has a π -rotation symmetry about the z axis: $x \rightarrow -x$, $y \rightarrow -y$, $z \rightarrow z$, and $u \rightarrow -u$.

A. Methods

Throughout this study, we choose $a = 6$ and $b = 0.1$ for which the system has three coexisting attractors: a torus (quasi-periodic behavior) and two chaotic attractors. The large difference in a and b gives rise to a large separation of time scales as will be seen in Sec. IV below. For future reference, the Lyapunov exponents for the chaotic attractors are $(0.2520, 0, -0.0052, -1.2467)$.¹⁴ The inverse of the largest positive exponent, equal to 3.97 time units, is known as the Lyapunov time and gives the characteristic exponential growth time constant of small differences in the attractor trajectories.

We generate synthetic data by numerically integrating Eq. (1) using a variable step size fourth-order Runge–Kutta method (RK45, SciPy solve_ivp) with an absolute error tolerance of 10^{-7} . We save the data at regular steps with $dt = 0.05$ whose value selection is discussed below.

Run times given below are for running Python 3.9, Numpy 1.20.3, SciPy 1.7.1, and Scikit learn 0.24.2 on an Intel i7-12700 processor running at 2.1 GHz.

III. NEXT-GENERATION RESERVOIR COMPUTING

The task of the NG-RC is to learn the flow $\mathbf{F}(\mathbf{x}_n)$ of the dynamical system using discretely sampled training data, here based on synthetic data generated by integrating Eqs. (4)–(7). Learning the flow has the advantage that it may speed up overall prediction time when dt is taken larger than the typical time step needed to accurately solve numerically the differential equations. This speedup is important for control applications and may find application in speeding up simulations of systems displaying spatiotemporal chaos.¹⁵

Briefly, the NG-RC expresses the flow as

$$\mathbf{F}(\mathbf{x}_n) = \mathbf{x}_n + \mathbb{W}_{out} \mathbf{O}_{total,n}(\mathbf{x}_n, \mathbf{x}_{n-1}, \dots, \mathbf{x}_k), \quad (8)$$

where $\mathbb{W}_{out}$ is the set of trainable weights of the output layer and $\mathbf{O}_{total,n}$ is a nonlinear feature vector described below that depends on the current state of the system $\mathbf{x}_n$ and $(k-1)$ past states. Equations (2) and (8) describe a discrete dynamical system that has memory, which is controlled by $\mathbb{W}_{out}$ and k . To be a universal dynamical system approximator,^{11,16,17} we must take the limit $k \rightarrow \infty$. However, it is found that the NG-RC can give highly accurate predictions even when k is small, which depends on dt . For example, when dt is much smaller than the shortest characteristic time of the system, $k=1$ is usually large enough and the highly weighted terms in $\mathbf{O}_{total,n}$ are largely determined by $\mathbf{f}$. For larger dt , other terms increase in importance but the nonlinear terms in $\mathbf{O}$ can be motivated by the vector field.

The feature vector is a concatenation of a constant term c , a linear part $\mathbf{O}_{lin}$, and a nonlinear part $\mathbf{O}_{nonlin}$. The $(d \cdot k + 1)$ -dimension linear part is given by

$$\mathbf{O}_{lin,n} = [\mathbf{x}_n^T, \mathbf{x}_{n-1}^T, \dots, \mathbf{x}_k^T]^T, \quad (9)$$

and, hence, includes $(k-1)$ time-delay pre-images of the current state of the system.

Choosing the nonlinear part requires some skill based on insights gained from the vector field if it is known. If there is no knowledge about the system, using a radial basis function,¹⁸ for example, might be appropriate. However, we find that a low-order polynomial function often works well even when the vector field has more complex functions.⁹ Here, Eqs. (4)–(7) only contain constant, linear, and quadratic terms and this motivates taking

$$\mathbf{O}_{nonlin,n} = [\mathbf{O}_{lin,n} [\otimes] \mathbf{O}_{lin,n}]^T. \quad (10)$$

The operator $[\otimes]$ performs the outer product between the two vectors, selects the unique quadratic monomials, flattens the data structure, and has dimension $(d \cdot k)(d \cdot k + 1)/2$. The total feature vector is then given by

$$\mathbf{O}_{total,n} = [c, \mathbf{O}_{lin,n}^T, \mathbf{O}_{nonlin,n}^T]^T, \quad (11)$$

which has dimension $1 + (d \cdot k) + (d \cdot k)(d \cdot k + 1)/2$.

We mention that reservoir computers realized with linear and passive photonic circuits that provide delay combined with a quadratic nonlinearity in the optical-to-electrical conversion in the output layer have been applied successfully to a variety of tasks.^{19–21} However, the form of the nonlinearity cannot be changed easily, the input data may need time-dependent masking,²¹ and there is no direct theoretical connection between these reservoirs and a traditional RC.

Because the NG-RC has memory due to the time-delay terms, prediction using Eqs. (2) and (8) requires data from $(k-1)$ past steps. Thus, some data must be sacrificed to “warm up” the model. For the NG-RC, this is only $(k-1)$ data points, whereas it is typically 10^3 – 10^5 data points for a traditional RC (10^3 was used in Ref. 13). As seen below, this is important for accurate learning the co-existing attractors of Eqs. (4)–(7) and for reproducing the basin of attraction as discussed in Sec. IV E.

A. The forecasting task

Our goal is to learn the one-step-ahead mapping of the dynamical system given by Eq. (2). This involves two phases: (1) a training phase based on supervised learning using time-series data produced by the system and (2) a forecasting phase where the output of the learned model is fed back to its input, thus realizing an autonomous system that should reproduce the characteristics of the learned system.

1. Training phase

Training the NG-RC model entails collecting a block of N_{train} contiguous data points, which is used to create a matrix $\mathbf{Y}$ containing the difference of the state of the system at each step $\mathbf{Y}_{n+1} = \mathbf{x}_{n+1} - \mathbf{x}_n$ over this time. This definition is motivated by Eq. (3), where we seek to learn the integral on the right-hand side of the equation. We combine this block of data with a corresponding feature-vector block $\mathbf{O}_{total}$. The trained output weight matrix is obtained using regularized regression,

$$\mathbb{W}_{out} = \mathbf{Y} \mathbf{O}_{total}^T (\mathbf{O}_{total} \mathbf{O}_{total}^T + \alpha \mathbb{I})^{-1}, \quad (12)$$

where α is the ridge regression parameter that prevents overfitting and $\mathbb{I}$ is the identity matrix.

2. Forecasting phase

After training, the model of the learned system is given by Eqs. (2) and (8) and no external data are fed into the model. From Eq. (8), we see that we require $(k-1)$ past states of $\mathbf{x}$ and, thus, we need to provide k initial conditions of $\mathbf{x}$. These data can be provided by a few experimental observations or evaluations of Eq. (1). Often, these data are already available if forecasting commences immediately after training. A traditional RC faces a similar requirement in that the reservoir needs to be “warmed up” before it can begin to make predictions and this warmup time can be substantial as mentioned above.

B. Comparing actual and forecasted attractors

Following our previous work,¹³ we use a set of metrics for determining the difference between the actual and forecasted attractors, so we can make a direct comparison between the performance of the NG-RC and an implementation of a traditional RC. For both attractors, we find the time average for each variable $\langle v \rangle$ and the time average of the absolute value of the variables $\langle |v| \rangle$ with $(v = x, y, z, u)$ evaluated over 5000 time units.

The differences between the average (absolute) forecasted v and the ground-truth $\tilde{v}$, normalized by the absolute ground-truth averages are given by

$$\Delta_v \equiv \frac{\langle v \rangle - \langle \tilde{v} \rangle}{\langle |\tilde{v}| \rangle}, \quad (13)$$

$$\Delta_{|v|} \equiv \frac{\langle |v| \rangle - \langle |\tilde{v}| \rangle}{\langle |\tilde{v}| \rangle}. \quad (14)$$

Here, Δ_v is a measure of the displacement of the center of mass of the attractors along each axis, while $\Delta_{|v|}$ is a measure of the difference of the extent of the attractors along each axis. For the attractor j with $j = (t = \text{torus}, c- = \text{chaotic attractor with } u < 0, c+ = \text{chaotic attractor with } u > 0)$, we define the metric

$$\Delta_{att,j} \equiv \sqrt{\sum_i (\Delta_i^2 + \Delta_{|i|}^2)}. \quad (15)$$

To quantify the overall success of the NG-RC model for forecasting over all three attractors, we define a total error metric given by

$$\Delta_{tot} \equiv \sqrt{\sum_j \Delta_{att,j}^2}. \quad (16)$$

These metrics only give a sense of the forecasting accuracy of the NG-RC model but allow us to directly compare to our results reported in Röhmer *et al.*¹³

IV. RESULTS

In this study, we learn the dynamics of the torus solution to Eq. (1) and then use this learned model to forecast the torus beyond the training data set to demonstrate the generalization of the model. We then use the same model to predict the two symmetric chaotic attractors by changing the initial conditions. This approach is similar to that taken previously in Röhmer *et al.*,¹³ where the training of the traditional RC used data from one of the chaotic attractors and the torus and symmetric attractors were forecasted. Learning on the torus as done here is more challenging because a much smaller region of phase space is explored by the trajectory during training, and, hence, it is believed that the model will be less accurate. Nevertheless, we show accurate predictions for all attractors.

Finally, we use the trained model to predict a section of the basin of attraction and show that it is also reproduced with high accuracy.

A. Training on the torus

We start by displaying the temporal evolution of the variables of Eqs. (4)–(7). The large difference in the model parameters a and b results in a large separation of timescales. Figure 1 shows the initial transient of the dynamics on a fine scale for initial conditions that generate the torus. The dynamical system exhibits fast oscillations, with the fastest behavior in the z -variable on a time scale of ~ 2.2 time units. In addition, we find also a much slower timescale in the variation of the waveform envelope, being most apparent for the u -variable.

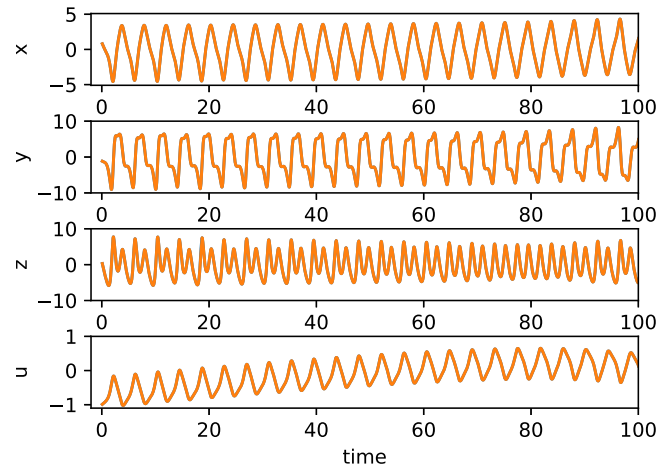


FIG. 1. Transient dynamics of the system with initial conditions $[1, -1, 1, -1]$. The ground-truth data are shown in blue and the NG-RC model prediction is in orange; they overlay accurately, and, hence, the blue trace is not visible.

Figure 2 shows the transient dynamics on a longer timescale. From the envelope of the u -variable, we see that the period of the slow oscillation is ~ 150 time units, which is $\sim 68\times$ slower than the fastest time scale. The slower dynamics are also apparent in the envelopes of the other variables but have a smaller effect in comparison to the z -variable.

In the RC literature using continuous dynamical systems as reservoirs, it has been observed that the best prediction ability is obtained when adjusting the reservoir fading memory timescale to be similar to the learned dynamics.²² Given the large difference in timescale of this system, it is not obvious how to best design the reservoir: should the reservoir have nodes with a range of timescales

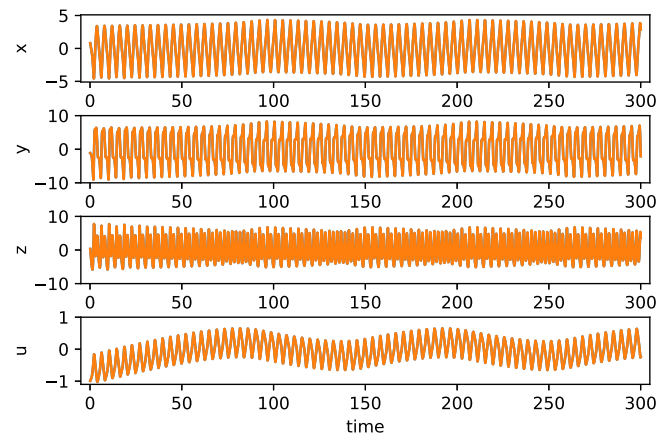


FIG. 2. Transient dynamics of the system on a longer timescale with initial conditions $[1, -1, 1, -1]$. The ground-truth data are shown in blue and the NG-RC model prediction is in orange; they overlay accurately, and, hence, the blue trace is not visible.

covering both seen in Fig. 2? Or should there be a single node timescale to reduce the number of metaparameters that must be optimized as used in Röhmer *et al.*?¹³

In the NG-RC approach, the situation is clearer. Our goal is to learn the flow of the dynamical system and this flow encodes both the slow and fast timescales. We select dt to be shorter than the fastest time scale (e.g., the sharper spike-like behavior seen in the y -variable of Fig. 1), but we need to train the model over a sufficiently long time period so that the NG-RC experiences the slower dynamics. Here, we take $dt = 0.05$ and the training time equal to 300 time units ($N_{\text{train}} = 6000$). The data shown in Fig. 2, including the transient, are used to train the NG-RC model.

Two of the few remaining metaparameters are the values of k and α . The value of k depends on the step size dt . For small dt (much less than the characteristic time scale of the fastest temporal feature), $k = 1$ typically works well, which can be motivated by considering the Euler method for integrating differential equations; here, only the vector field appears in the integration method, which corresponds to taking $k = 1$. Increasing k only marginally improves the NG-RC testing performance. For moderate dt , as used here, $k = 1$ typically exhibits higher error and there can be substantial improvement using $k = 2$. This is analogous to reducing the error in numerical integrators by using data from multiple time steps. Going to $k = 3$ or larger gives only modest improvement while the increase in feature-vector size has a computational penalty.

For the system considered here, we explore values of k up to three and find that $k = 2$ gives good results for both predicting the torus and the chaotic attractors. For each value of k , we use a coarse grid search to identify the optimized value of α ; the performance is good over a wide range of the ridge regression parameter and, hence, the search proceeds quickly. When taking $k = 3$, the testing performance on the torus is somewhat better, but there is little improvement in the accuracy of predicting the chaotic attractors (see below). Hence, we use $k = 2$ in our studies for which $\alpha = 4 \times 10^{-5}$ is the optimal value.

The agreement between the ground-truth data generated by Eq. (1) and the NG-RC model prediction during the training phase is excellent. These data are overlaid in the figures and their difference cannot be observed on this scale. We find that the normalized root mean square error (NRMSE) training error is 1.5×10^{-5} .

Regarding the training computational time using the computer specified in Sec. II A, we find that creating the feature vector is the most time consuming operation, taking ~ 145 ms, whereas finding $\mathbb{W}_{\text{out}}$ only takes ~ 2 ms. Speeding up feature vector creation might be achieved by using the polynomial regression methods available in Scikit learn.

B. Testing on the torus

To test the generalization of the trained NG-RC model, we use it to forecast the dynamics for another 150 time units beyond the training time, which is behavior not seen during training. Figure 3 shows the ground-truth data and NG-RC predictions, which overlay entirely on this scale. The NRMSE for these data is 3.3×10^{-3} , indicating high-quality generalization.

The error metrics for the torus testing phase (using 5000 time units testing time) are given in Table I. We see that the largest error

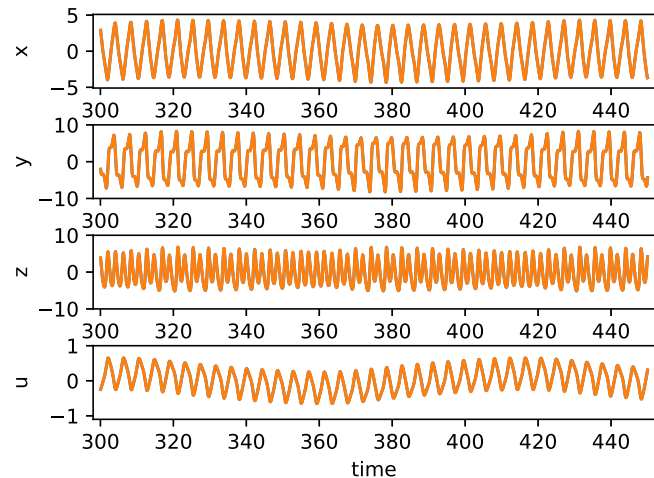


FIG. 3. Testing the NG-RC model on the torus dynamics. Ground-truth data are shown in blue and the model prediction is in orange; they overlay accurately, and, hence, the blue trace is not visible. The initial condition for the model is taken from the last $k = 2$ data points from the training data set.

is in the u -variable where $\Delta_u \sim 1\%$. The errors for the other variables are significantly smaller. For the torus attractor, we find that $\Delta_{\text{att},t} = 1.2 \times 10^{-2}$, largely dominated by Δ_u . For context, in Röhmer *et al.*¹³ we found $\Delta_u \sim 0.7$ for the torus, over a factor of $58\times$ larger than we obtain here, although in Ref. 13, we trained on one of the chaotic attractors rather than training on the torus as we do here.

In Secs. IV C and IV D, we used $\mathbb{W}_{\text{out}}$ found from training on the torus, which obeys the symmetry of the system. Then, we use two initial conditions that are obtained by the symmetry operation within the system.

C. Testing on the chaotic attractor with $u < 0$

We now use the identical trained NG-RC model but for initial conditions expected to converge toward the chaotic attractor with the average value of $u < 0$, specifically $[0, 4, 0, -5]$. This choice of initial condition is also used for generating the basin of attraction discussed in Sec. IV E below following the procedure used in Ref. 14. We need to provide the initial condition and $(k - 1) = 1$ (with $k = 2$ as we use here) additional points from the Li-Sprout model¹⁴ to initialize the NG-RC. We generate the one additional data point at time dt beyond the initial condition by using the ground-truth model given by Eq. (1) but then use the NG-RC learned autonomous model thereafter.

TABLE I. Error metrics for the torus forecasting phase.

v	Δ_v	$\Delta_{ v }$
x	7.1×10^{-4}	9.9×10^{-5}
y	3.2×10^{-4}	-6.7×10^{-5}
z	-5.6×10^{-5}	-4.7×10^{-4}
u	1.1×10^{-2}	-5.4×10^{-3}

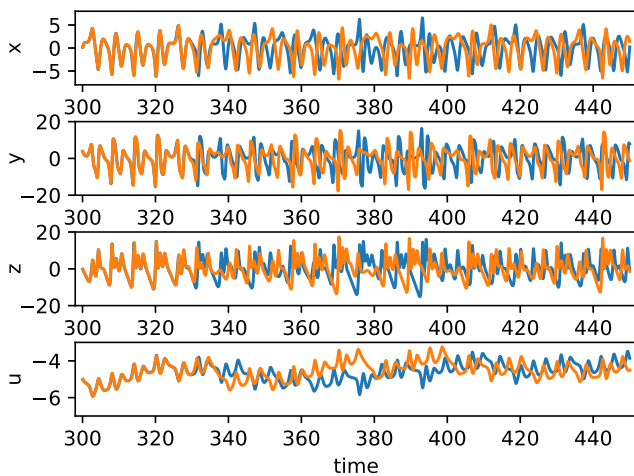


FIG. 4. Testing the NG-RC model on the chaotic dynamics where $u < 0$. The initial condition is $[0, 4, 0, -5]$. Ground-truth data are shown in blue and the model prediction is in orange.

Figure 4 compares ground truth with NG-RC model prediction. We see that the trajectories well overlap for ~ 30 time units (from 300 to 330 time units), which is ~ 7.6 Lyapunov times. The trajectories necessarily diverge because of tiny model errors and the chaotic nature of the dynamics. Forecasting for such a long time is near the state-of-the-art for machine learning algorithms and especially stands out given the large separation of time scales in the problem, which has not been explored previously to any great extent because of the perceived hardness of this problem.

Beyond 330 time units, the forecasted dynamics is consistent with the chaotic attractor. This is quantified using the error metrics given in Table II. We see that all errors are below 0.4% and the error in the u -variable metrics is smaller than for the torus. The overall error metric for this attractor is $\Delta_{att,c-} = 8.2 \times 10^{-3}$.

D. Testing on the chaotic attractor with $u > 0$

We also forecast the second symmetric chaotic attractor by taking the initial condition $[0, -4, 0, 5]$. Figure 5 shows the temporal evolution of NG-RC-forecasted behavior of both chaotic attractors. It is seen that they initially agree with each other for ~ 30 time units, but with the expected symmetry relations. We do not enforce this symmetry in the NG-RC feature vector, but we see that the NG-RC model properly learns the symmetry.^{15,23}

TABLE II. Error metrics for the chaos forecasting phase with $u < 0$.

v	Δ_v	$\Delta_{ v }$
x	3.9×10^{-4}	-1.1×10^{-3}
y	-1.9×10^{-4}	-1.2×10^{-3}
z	1.1×10^{-3}	6.1×10^{-3}
u	3.7×10^{-3}	-3.7×10^{-3}

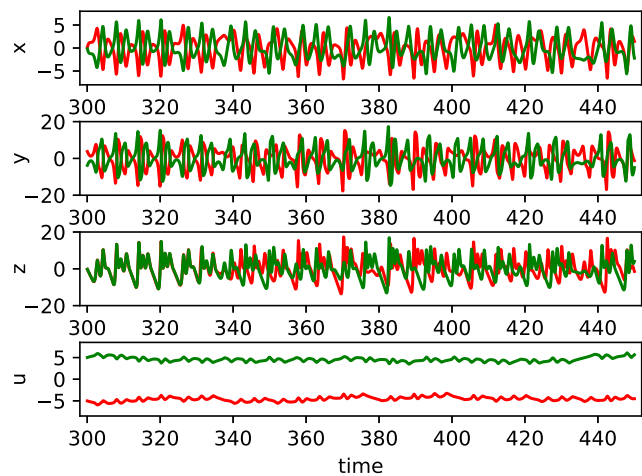


FIG. 5. Testing the NG-RC model on the chaotic dynamics for both chaotic attractors. Using initial condition $[0, -4, 0, 5]$, the trajectory goes to the strange attractor with $u > 0$ (green lines), whereas it goes to the attractor with $u < 0$ (red lines) using the initial condition $[0, 4, 0, -5]$.

Beyond 330 time units, the trajectories diverge because of the chaotic nature of the attractors, but the dynamics appears qualitatively to be part of the same attractor, which is verified by the error metrics given in Table III. Here, the error in the u -variable is somewhat larger than for the other chaotic attractor but is still only $\sim 1\%$. The overall attractor error metric is $\Delta_{att,c+} = 1.9 \times 10^{-2}$ dominated by the error in the u -variable. The differences in the error metrics appearing in Tables II and III are largely due to the length of the testing data set size.

E. Forecasting the basin of attraction

Given the high prediction quality found in the sections above, we are motivated to see how well the trained NG-RC model predicts the basin of attraction. The basin is complex with fractal structure¹⁴ and potentially even riddled structure,²⁴ where only a small change in initial conditions drives the system to different attractors. We find the basin by selecting the initial conditions $[x_0, y_0, z_0, u_0]$ with $y_0 = u_0 = 0$ and $x_0 \in [-3, -1]$ or $x_0 \in [1, 3]$ and $z_0 \in [5, 10]$, augmenting the NG-RC model with one additional data point at time dt generated by the Li and Sprott¹⁴ model [Eq. (1)], and integrating for 200 time units. If $u < -2$ ($u > 2$) at this time, then the point is associated with the chaotic attractor with $u < 0$ ($u > 0$), or

TABLE III. Error metrics for the chaos forecasting phase with $u > 0$.

v	Δ_v	$\Delta_{ v }$
x	2.7×10^{-4}	4.3×10^{-4}
y	2.5×10^{-4}	-4.0×10^{-3}
z	3.9×10^{-3}	-7.7×10^{-3}
u	-1.1×10^{-2}	-1.1×10^{-2}

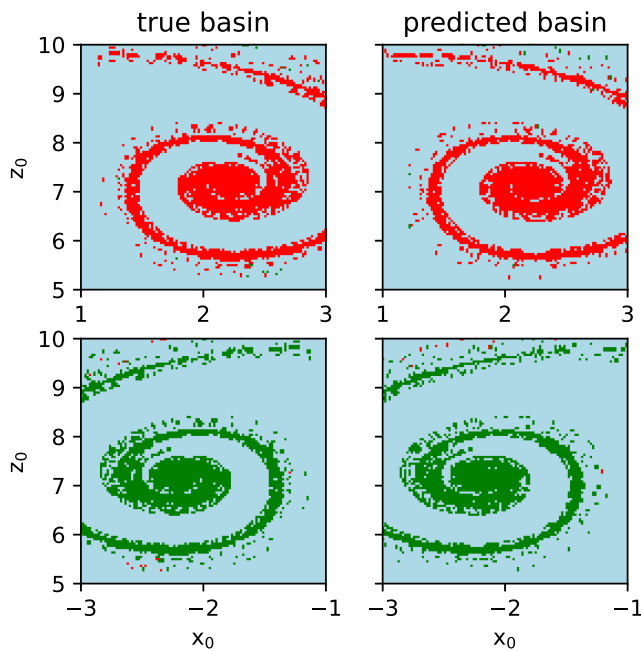


FIG. 6. The basin of attraction from the prediction of Eq. (1) and the trained NG-RC model evaluated on a (100×100) grid. The basin for the $u > 0$ ($u < 0$) chaotic attractor is shown in red (green) while the torus basin of attraction is shown as pale blue following the color scheme of Li and Sprott.¹⁴

with the torus otherwise. We use the same procedure both with the ground-truth model and the trained NG-RC model.

We zoom in on a small region of the basin of attraction shown in Fig. 11 of Li and Sprott¹⁴ to highlight its fractal structure. Figure 6 shows the ground-truth basins on the left and the NG-RC-forecasted basins on the right for two different regions that highlight the basin symmetry. As mentioned above, we stress that we do not force this symmetry in the NG-RC feature vector; it learns it naturally from the training data.

To quantify the accuracy of the NG-RC model reproducing the basins of attraction, we determine the fraction of pixels (10^4 total pixels) painted correctly for the two chaotic attractors. For the chaotic attractor with $u < 0$ ($u > 0$), we find that 90.0% (90.2%) of the pixels are painted correctly, indicating high-accuracy basin prediction. Our ability to generate the basin of attracting is enabled by the NG-RC having a short warmup time as discussed in II A: we only require an additional $(k - 1)$ points generated by the ground-truth dynamics for the data shown in Fig. 6 as discussed in Sec. IV C above.

In our previous work¹³ using a traditional RC, 1000 data points from Eq. (1) were used to warm up the reservoir, corresponding to 200 time units (step size of 0.2 time units). The system will have already approached the final attractor in this time so the traditional RC model we used previously cannot be used to determine the basin of attraction. Long warm-up times are present in all recurrent neural networks except for the NG-RC, which appears to use the minimum memory time possible and still achieve high prediction accuracy.

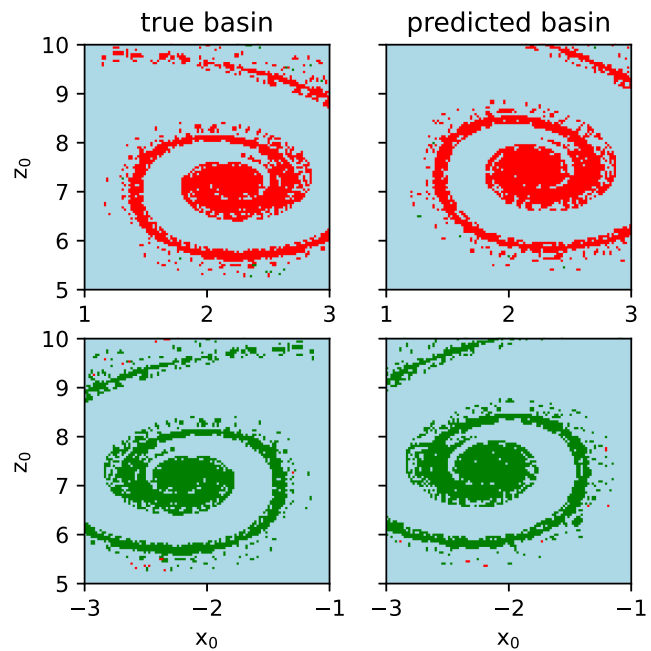


FIG. 7. The basin of attraction from the prediction of Eq. (1) and the trained NG-RC model evaluated on a (100×100) grid using the bootstrapping method. The color scheme is the same as in Fig. 6.

Next, we show that even the warmup points in the NG-RC can be removed using a bootstrapping method. For simplicity, we assume $k = 2$ as used throughout this work. We generate an auxiliary NG-RC model with $k = 1$ (no memory in the model) and use it to generate the one required additional data point beyond the initial condition $[x_0, y_0, z_0, u_0]$. The initial condition $[x_0, z_0]$ and this additional data point generated by the auxiliary NG-RC model with $k = 1$ are the only data needed to evaluate the $k = 2$ NG-RC model.

Figure 7 shows the ground-truth and predicted basin of attraction using the bootstrapping method. Qualitatively, the attractor basins appear similar with higher error in the top of the plots evident, but they continue to display the fractal or riddled structure. For the chaotic attractor with $u < 0$ ($u > 0$), we find that 80.8% (79.8%) of the pixels are painted correctly, indicating continued high-accuracy basin prediction. The slightly lower accuracy represents the cost of not using any ground-truth data to warm up the NG-RC. Clearly, the bootstrapping method can be generalized for any k using $(k - 1)$ auxiliary NG-RC models for generating the warmup data.

V. DISCUSSION

We emphasize in this work that a reservoir computer trained on discretely sampled data learns the underlying flow. For a smooth and simple vector field such as that in Eq. (1), we expect that the flow is also smooth. Thus, learning the flow in one region of phase space allows us to predict dynamics in other regions of phase space²⁵ as long as the learned model is accurate.

We expect all machine learning algorithms to fail to extrapolate to other regions of phase space if the vector field is discontinuous, such as in a system with a border collision²⁶ or other non-analytic vector fields. We also expect that the extrapolation will be more difficult for an analytic continuous flow if the associated vector field has high-order polynomials, exponential functions, or bump functions that dominate in some regions of phase space different from where learning is performed. Likely, criteria for learning flows and unseen attractors will require considering analytic functions and a bound on the learning accuracy, which we will explore in the future.

The NG-RC model is particularly simple because the vector field has only quadratic nonlinear terms, and, hence, we expect the flow to be dominated by similar terms and it has finite memory.⁹ Here, $\mathbb{W}_{out}$ has shape (4×45) , resulting in 180 trainable parameters. In contrast, $\mathbb{W}_{out}$ has shape (4×300) for the traditional RC of Röhm *et al.*,¹³ resulting in 1200 trainable parameters. Having fewer trainable parameters is associated with smaller training data set sizes.⁹ Also, solving the regression problem [Eq. (12)] scales with the length of the training data set size and quadratically on the number of feature-vector components; thus, we anticipate a speedup of ~ 75 for this part of the training.

The error metric over all three attractors for the trained NG-RC model is $\Delta_{tot} = 2.4 \times 10^{-2}$ or 2.4%. Using a traditional RC, Röhm *et al.*¹³ finds $\Delta_{tot} = 2.4$. Thus, the trained NG-RC model is $100\times$ more accurate. We do not understand in detail why the traditional RC did not perform with as low of an error. It is worth noting that the traditional RC was modeled as a continuous-time echo state network rather than the discrete-time model considered here and that may cause some differences. Also, the ground-truth model was sampled using a sample time of 0.2 time units, whereas we use 0.05 here. Another issue is that the traditional RC has many more meta-parameters, which takes considerable computation time to optimize and there could be a small region of parameter space that gives better accuracy and was missed. In Ref. 13, we also kept the decay rate of each reservoir node fixed to reduce the number of parameters needed to be optimized; this might be a source of the lower accuracy of the traditional RC but there are likely other factors as well.

We also find that the NG-RC model can account well for disparate time scales. The key is to take dt smaller than the fastest timescale but take the training time longer than the slowest timescale.

One open question that our study does not address is whether the performance of a traditional RC combined with delayed inputs or outputs^{27–29} can outperform the simple NG-RC explored here. Another question is whether the NG-RC can learn the phase-space structure of higher-dimensional systems; studies on spatiotemporal systems are already underway.¹⁵ Also, we will explore using adapted basis functions or functions that focus on a region of phase space to interpolate between discontinuous regions.

ACKNOWLEDGMENTS

D.J.G. gratefully acknowledges financial support of the Air Force Office of Scientific Research, Contract #FA9550-20-1-0177. I.F. and A.R. acknowledge the Spanish State Research Agency through the Severo Ocha and María de Maeztu Program for Centers and Units of Excellence in R&D (MDM-2017-0711) funded by

MCIN/AEI/10.13039/501100011033. A.R. is currently an International Research Fellow of the Japan Society for the Promotion of Science (JSPS).

AUTHOR DECLARATIONS

Conflict of Interest

D.J.G. is a co-founder of ResCon Technologies LLC, which is commercializing reservoir computing.

Author Contributions

Daniel J. Gauthier: Conceptualization (equal); Data curation (lead); Formal analysis (equal); Funding acquisition (lead); Investigation (lead); Methodology (lead); Project administration (lead); Software (lead); Validation (lead); Visualization (lead); Writing – original draft (lead); Writing – review & editing (equal). **Ingo Fischer:** Conceptualization (equal); Formal analysis (equal); Methodology (supporting); Writing – review & editing (equal). **André Röhm:** Conceptualization (equal); Formal analysis (equal); Methodology (supporting); Writing – review & editing (equal).

DATA AVAILABILITY

The data that support the findings of this study are openly available in Zenodo at <https://doi.org/10.5281/zenodo.7241639>, Ref. 30.

REFERENCES

- ¹M. W. Grieves, “Virtually intelligent product systems: Digital and physical twins,” in *Complex Systems Engineering: Theory and Practice* (American Institute of Aeronautics and Astronautics, Inc., 2019), pp. 175–200.
- ²H. Jaeger and H. Haas, “Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication,” *Science* **304**, 78–80 (2004).
- ³W. Maass, T. Natschläger, and H. Markram, “Real-time computing without stable states: A new framework for neural computation based on perturbations,” *Neural Comput.* **14**, 2531–2560 (2002).
- ⁴P. R. Vlachos, J. Pathak, B. R. Hunt, T. P. Sapsis, M. Girvan, E. Ott, and P. Koumoutsakos, “Backpropagation algorithms and reservoir computing in recurrent neural networks for the forecasting of complex spatiotemporal dynamics,” *Neural Netw.* **126**, 191–217 (2020).
- ⁵S. Bompas, B. Georgeot, and D. Guéry-Odelin, “Accuracy of neural networks for the simulation of chaotic dynamics: Precision of training data vs precision of the algorithm,” *Chaos* **30**, 113118 (2020).
- ⁶J. Pathak, S. Subramanian, P. Harrington, S. Raja, A. Chattopadhyay, M. Mardani, T. Kurth, D. Hall, Z. Li, K. Azizzadenesheli, P. Hassanzadeh, K. Kashinath, and A. Anandkumar, “FourCastNet: A global data-driven high-resolution weather model using adaptive Fourier neural operators,” *arXiv:2202.11214* (2022).
- ⁷T. Arcomano, I. Szunyogh, A. Wikner, J. Pathak, B. R. Hunt, and E. Ott, “A hybrid approach to atmospheric modeling that combines machine learning with a physics-based numerical model,” *J. Adv. Model. Earth Syst.* **14**, e2021MS002712 (2022).
- ⁸“*Reservoir Computing: Theory, Physical Implementations, and Applications*,” edited by K. Nakajima and I. Fischer (Springer Nature, Singapore, 2021).
- ⁹D. J. Gauthier, E. Bollt, A. Griffith, and W. A. S. Barbosa, “Next generation reservoir computing,” *Nat. Comm.* **12**, 5564 (2021).
- ¹⁰M. S. Goldman, “Memory without feedback in a neural network,” *Neuron* **61**, 621–634 (2009).
- ¹¹S. Boyd and L. O. Chua, “Fading memory and the problem of approximating nonlinear operators with Volterra series,” *IEEE Trans. Cir. Syst. CAS-32*, 1150–1161 (1985).

- ¹²E. Bollt, “On explaining the surprising success of reservoir computing forecaster of chaos? The universal machine learning dynamical system with contrast to VAR and DMD,” *Chaos* **31**, 013108 (2021).
- ¹³A. Röhm, D. J. Gauthier, and I. Fischer, “Model-free inference of unseen attractors: Reconstructing phase space features from a single noisy trajectory using reservoir computing,” *Chaos* **31**, 103127 (2021).
- ¹⁴C. Li and J. C. Sprott, “Coexisting hidden attractors in a 4-D simplified Lorenz system,” *Int. J. Bifurcation Chaos* **3**, 1450034 (2014).
- ¹⁵W. A. S. Barbosa and D. J. Gauthier, “Learning spatiotemporal chaos using next-generation reservoir computing,” *Chaos* **32**, 093137 (2022).
- ¹⁶L. Grigoryeva and J. P. Ortega, “Universal discrete-time reservoir computers with stochastic inputs and linear readouts using non-homogeneous state-affine systems,” *J. Mach. Learn. Res.* **19**, 1–19 (2018).
- ¹⁷L. Gonon and J. P. Ortega, “Reservoir computing universality with stochastic inputs,” *IEEE Trans. Neural Networks Learn. Syst.* **31**, 100–112 (2020).
- ¹⁸B. Schölkopf and A. J. Smola, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond* (The MIT Press, Cambridge, MA, 2002).
- ¹⁹K. P. Jackson, S. A. Newton, B. Moslehi, M. Tur, C. C. Cutler, J. W. Goodman, and H. J. Shaw, “Optical fiber delay-line signal processing,” *IEEE Trans. Microw. Theory Tech.* **MTT-33**, 193–210 (1985).
- ²⁰K. Vandoorne, P. Mechet, T. V. Vaerenbergh, M. Fiers, G. Morthier, D. Verstraeten, B. Schrauwen, J. Dambre, and P. Bienstman, “Experimental demonstration of reservoir computing on a silicon photonics chip,” *Nat. Commun.* **5**, 3541 (2014).
- ²¹Q. Vinckier, F. Duport, A. Smerieri, K. Vandoorne, P. Bienstman, M. Haelterman, and S. Massar, “High-performance photonic reservoir computer based on a coherently driven passive cavity,” *Optica* **2**, 438–446 (2015).
- ²²D. Canaday, A. Griffith, and D. J. Gauthier, “Rapid time series prediction with a hardware-based reservoir computer,” *Chaos* **28**, 123119 (2018).
- ²³W. A. S. Barbosa, A. Griffith, G. E. Rowlands, L. C. G. Govia, G. J. Ribeill, M.-H. Nguyen, T. A. Ohki, and D. J. Gauthier, “Symmetry-aware reservoir computing,” *Phys. Rev. E* **104**, 045307 (2021).
- ²⁴J. Sommerer and E. Ott, “A physical system with qualitatively uncertain dynamics,” *Nature* **365**, 138–140 (1993).
- ²⁵M. Hara and H. Kokubu, “Learning dynamics by reservoir computing (in memory of Prof. Pavol Brunovský),” *J. Dyn. Diff. Equat.* (published online, 2022).
- ²⁶C. M. Berger, X. Zhao, D. G. Schaeffer, W. Krassowska, H. M. Dobrovolny, and D. J. Gauthier, “Period-doubling bifurcation to alternans in paced-cardiac tissue: Crossover from smooth to border-collision characteristics,” *Phys. Rev. Lett.* **99**, 058101 (2007).
- ²⁷Y. Sakemi, K. Morino, T. Leleu, and K. Aihara, “Model-size reduction for reservoir computing by concatenating internal states through time,” *Sci. Rep.* **10**, 21794 (2020).
- ²⁸T. L. Carroll, “Adding filters to improve reservoir computer performance,” *Physica D* **416**, 132798 (2021).
- ²⁹L. Jaurigue, E. Robertson, J. Wolters, and K. Lüdge, “Reservoir computing with delayed input for fast and easy optimisation,” *Entropy* **23**, 1560 (2021).
- ³⁰D. J. Gauthier (2022). “quantinfo/learning-unseen-attractors-paper-code: Version used in the published paper,” Zenodo. <https://doi.org/10.5281/zenodo.7241639>.